

# Flutter Lab Exam Pack

---

Your question set has 10 possible questions. Questions 1-3 are setup/demo questions; questions 4-10 are Flutter programs. The fastest way to prepare is:

1. Install Flutter through VS Code tonight.
2. Run one program (start with `q09_stateful_counter.dart`) in Chrome.
3. Practise typing the common skeleton, then one layout program and the counter without looking.
4. Read the viva answers once before sleeping.

## 1. Install Flutter on this Mac using VS Code

---

This Mac is Apple Silicon and already has VS Code plus Xcode Command Line Tools. Flutter itself is not yet installed.

1. Open VS Code and install the **Flutter** extension. It installs Dart support as well.
2. Press `Cmd+Shift+P` and choose **Flutter: New Project**.
3. When VS Code asks for the SDK, choose **Download SDK**.
4. Choose a location you will remember, such as `/Users/<your-name>/development`. Do not put it inside a temporary folder.
5. Choose **Add SDK to PATH**. Close and reopen VS Code and Terminal when the download completes.
6. In Terminal, run `flutter doctor`. For tonight, it is enough if Flutter, VS Code, and Chrome/Web are green. Android Studio and a phone simulator are optional for a browser run.
7. In VS Code, create an **Application** project named `flutter_lab`. Open `lib/main.dart`, replace it with one of the files in `code/`, save, then press `F5` and select Chrome.

Useful commands:

```
flutter doctor
flutter create flutter_lab
cd flutter_lab
flutter run -d chrome
```

Important correction for the wording in question 2: on macOS, use **VS Code** with the Flutter extension. **Visual Studio** is a different IDE and is not needed. You also do not install Dart separately for Flutter: the Flutter SDK includes the Dart SDK.

If you must run an Android emulator at college, install Android Studio, then Android SDK and an emulator. Run `flutter doctor` again and complete only the Android items it reports.

## 2. How to use the code files

---

Every `qXX_*.dart` file is a complete replacement for `lib/main.dart` in a standard Flutter project. They use only `material.dart`, so no package installation or `pubspec.yaml` change is needed.

For the image question, the app uses a public image URL and has an offline fallback icon, so it still runs if the classroom internet is unavailable.

### 3. The one Flutter skeleton to memorise

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('My App')),
        body: const Center(child: Text('Hello')),
      ),
    );
  }
}
```

Say this while writing it: `main` starts the app, `runApp` displays the root widget, `MaterialApp` gives Material design, `Scaffold` gives the screen structure, `AppBar` is the top bar, and `body` is the main screen area.

### 4. What each exam program proves

| Question | File                                    | What to say while demonstrating                                                                                                                             |
|----------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4        | <code>q04_core_widgets.dart</code>      | Text displays labels, Container styles its child, Center positions it, AppBar is the top bar, Scaffold is the page structure, and Image displays a picture. |
| 5        | <code>q05_row.dart</code>               | Row places children horizontally. Its main axis is horizontal; <code>spaceEvenly</code> distributes them.                                                   |
| 6        | <code>q06_column.dart</code>            | Column places children vertically. Its main axis is vertical.                                                                                               |
| 7        | <code>q07_stack.dart</code>             | Stack layers children; <code>Positioned</code> puts a child at an exact place.                                                                              |
| 8        | <code>q08_combined_layout.dart</code>   | The outer Column makes vertical sections, the Stack makes a layered banner, and the Row makes a horizontal action area.                                     |
| 9        | <code>q09_stateful_counter.dart</code>  | <code>setState</code> changes <code>count</code> and asks Flutter to rebuild the UI.                                                                        |
| 10       | <code>q10_stateless_profile.dart</code> | This screen has no changing data, so StatelessWidget is sufficient.                                                                                         |

## 5. Layout rules that prevent mistakes

---

- **Row:** left-to-right. `mainAxisAlignment` controls left/right spacing; `crossAxisAlignment` controls vertical alignment.
- **Column:** top-to-bottom. `mainAxisAlignment` controls up/down spacing; `crossAxisAlignment` controls horizontal alignment.
- **Stack:** children overlap. Put a `SizeBox` around it when you want a fixed drawing area. Use `Positioned` only inside Stack.
- **Container:** use `padding` for space inside; `margin` for space outside; `decoration` for color, border, and rounded corners.
- **Center:** centres one child in its available space.
- Use `const` when the widget has no runtime-changing value. It is a good habit, not mandatory for the exam.
- If a Column becomes taller than the screen, wrap it in `SingleChildScrollView`.

## 6. Stateless vs Stateful: the exam answer

---

| StatelessWidget                               | StatefulWidget                                               |
|-----------------------------------------------|--------------------------------------------------------------|
| UI does not change after it is built.         | UI can change while the app is running.                      |
| Has one <code>build()</code> method.          | Has a widget class plus a separate <code>State</code> class. |
| Examples: label, icon, static profile screen. | Examples: counter, checkbox, form field.                     |
| Does not call <code>setState()</code> .       | Calls <code>setState()</code> after changing state.          |

Use a StatefulWidget only when something on screen changes because of a tap, input, timer, or data update.

## 7. Ready-to-say answers for questions 1-3

---

### Q1. Git and VS Code setup

"Git tracks source-code changes and lets us restore or collaborate on versions. On macOS I install the Xcode Command Line Tools, which include Git, then verify with `git --version`. I install VS Code, add the Flutter extension, create a Flutter project, and verify the environment with `flutter doctor`."

### Q2. Flutter SDK setup and verification

"I install the Flutter extension in VS Code, use **Flutter: New Project**, choose **Download SDK**, choose an installation folder, and add the SDK to PATH. Then I restart VS Code and Terminal and run `flutter doctor`. It reports the Flutter SDK, Dart SDK, editor, and available devices."

### Q3. Dart SDK and VS Code role

"Dart is the language used to write Flutter apps. The Flutter SDK already includes the Dart SDK, so a separate Dart installation is unnecessary for Flutter. VS Code provides editing, Flutter/Dart extensions, debugging, hot reload, and the device selector."

## 8. Fast viva sheet

---

1. **What is Flutter?** Google's UI toolkit for building mobile, web, and desktop apps from one Dart codebase.
2. **What is a widget?** An immutable description of part of the UI. Everything displayed in Flutter is a widget.
3. **What is Dart?** The object-oriented, null-safe programming language used for Flutter code.
4. **Why Flutter?** One codebase, fast hot reload, rich widgets, and near-native performance.
5. **What does `build()` do?** It returns the widget tree that Flutter should draw for the current state.
6. **What is `BuildContext` ?** A widget's location in the widget tree; it lets Flutter find inherited information such as theme or navigator.
7. **What is `MaterialApp`?** The app-level Material Design widget; it supplies theme, routes, and navigation support.
8. **What is `Scaffold`?** A page layout with areas such as `appBar` , `body` , drawer, and floating action button.
9. **What is `AppBar`?** The top toolbar, usually containing a title and actions.
10. **What is hot reload?** It applies most code changes to the running app while preserving current state. Hot restart resets state.
11. **What is `flutter doctor` ?** A command that checks Flutter, IDEs, platform toolchains, and devices, then reports missing setup.
12. **What is `pubspec.yaml` ?** The project configuration file for app metadata, dependencies, assets, and fonts.
13. **What is `var` , `final` , and `const` ?** `var` can be reassigned; `final` is assigned once at runtime; `const` is a compile-time constant.
14. **What is null safety?** A variable cannot be null unless declared nullable, for example `String?` .
15. **List vs Set vs Map?** List is ordered and allows duplicates; Set has unique values; Map stores key-value pairs.
16. **What is `setState()` ?** A State method that tells Flutter state changed and the widget should rebuild.
17. **What are `mainAxisAlignment` and `crossAxisAlignment` ?** They align children along the main direction and the perpendicular direction.
18. **How do you show an image?** `Image.network` for a URL or `Image.asset` for an image declared in `pubspec.yaml` .
19. **Why use `Container`?** It combines size, padding, margin, alignment, and decoration around one child.
20. **What is Git used for?** Version control: tracking changes, branches, collaboration, and recovery.

## 9. 30-minute rescue plan

---

- **First 10 minutes:** install Flutter in VS Code and make `flutter doctor` work.
- **Next 10 minutes:** create one app, paste and run the Stateful counter, then change its title and button label using hot reload.
- **Next 5 minutes:** type Row, Column, and Stack code from memory. Remember their directions: Row = horizontal, Column = vertical, Stack = overlap.
- **Last 5 minutes:** say the 20 viva answers aloud, especially widget, Flutter, Dart, Scaffold, `setState`, hot reload, and `flutter doctor`.

## 10. During the lab

---

1. Read the selected question and state which widgets you will use.
2. Create a Flutter Application project, then replace `lib/main.dart`.
3. Save, select a device, and run. If the app is already running, use hot reload.
4. Demonstrate the exact requirement first. Do not add packages or advanced features.
5. If asked to explain, point to the relevant widget in the code and use the one-line explanations above.